

PD-3: Comments on events

Status: tickets approved · Revision 3 · Created by Ivan Blažević <ivan.blazevic@rubycode.co> · 30 September 2026

Overview

What

Signed-in people can leave a comment on an event page, to ask a question or share a link. Comments are listed oldest first, with the author's name and when it was posted. Authors can delete their own comments, and organizers can delete any comment on their event.

Why

Questions about events are asked in chat today and get lost. On the event page, the answer helps the next person with the same question.

Where

The event page, below the RSVP card.

Who

Team Gather. Ana Kovač reviews the pull requests.

Background

Existing Data Structure

These are the existing models, tables and files the change touches. The schema is SQLite (`db/schema.rb` , version `2026_09_29_214624`), and primary and foreign keys are `integer` .

Event — `app/models/event.rb` → `events`

- Columns: `id` `integer` not null, `title` `string` not null, `description` `text` nullable, `starts_at` `datetime` not null, `venue` `string` not null, `capacity` `integer` not null, `organizer_id` `integer` not null, `created_at` / `updated_at` `datetime` not null.
- Indexes: `index_events_on_organizer_id` , `index_events_on_starts_at` . Foreign key `events.organizer_id` → `users.id` .
- Associations: `belongs_to :organizer` , `class_name: "User"` , `has_many :rsvps` , `dependent: :delete_all` , `has_many :attendees` (going RSVPs only) `through: :rsvps` , `source: :user` .

- Relevant methods: `organized_by?(user)` returns true when `user.present? && organizer_id == user.id`. This is the only organizer check in the app, and it is used by `EventsController#require_organizer` and by the views. `rsvps_open?` is `starts_at.future?`.

Rsvp — `app/models/rsvp.rb` → `rsvps`

- Columns: `id`, `event_id` integer not null, `user_id` integer not null, `status` string not null (check constraint `status IN ('going', 'waitlisted')`), timestamps.
- Indexes: unique `(event_id, user_id)`, `(event_id, status, created_at)`, `user_id`. Foreign keys to `events` and `users`.
- Associations: `belongs_to :event`, `belongs_to :user`. Scope `in_line_order` orders by `created_at, id`.
- This model does not change. It is listed because the new section goes below the RSVP card, `app/views/events/_rsvp_card.html.erb`, and because its `event_id / user_id` columns, foreign keys and ordering are the pattern the new table follows.

User — `app/models/user.rb` → `users`

- Columns: `id`, `email_address` string not null (unique index), `password_digest` string not null, `name` string not null **default** `""`, timestamps.
- Associations: `has_many :sessions`, `dependent: :destroy`, `has_many :rsvps`, `dependent: :destroy`, `has_many :organized_events`, `class_name: "Event"`, `foreign_key: :organizer_id`, `dependent: :destroy`.
- Validates `name` presence. Because of the database default, rows created before that validation may still have an empty `name`.

Session — `app/models/session.rb` → `sessions`

- Columns: `id`, `user_id` integer not null, `ip_address`, `user_agent`, timestamps. `belongs_to :user`.
- Used by `app/controllers/concerns/authentication.rb` (`resume_session`, `require_authentication`, `authenticated?`) to set `Current.user`. Every controller requires sign-in unless it calls `allow_unauthenticated_access`.

Code the change touches

- `app/controllers/events_controller.rb`: `show` sets `@rsvp`, and sets `@rsvps` only for the organizer. `index` and `show` allow guests.
- `app/views/events/show.html.erb`: a two-column `.event__layout`. The left column is the `.event__description` card. The right column is the `aside#event-facts`, which holds the facts list, the `events/rsvp_card` partial, the "Edit event" link and, for the organizer, the `events/attendees` partial.
- `config/routes.rb`: `resources :events`, `except: :destroy` do `resource :rsvp`, `only: %i[ create destroy ]` end.
- There is no authorization library such as Pundit. Permission checks are model predicates like `Event#organized_by?`, called from controllers and views.

Architectural changes

Target design

Nothing in the schema stores comments today. This plan adds a new `Comment` model and `comments` table. A comment belongs to an `Event` and to the `User` who wrote it. It follows the same request pattern as RSVPs: a nested, resourceful controller submits a normal form or `button_to`, then redirects back to the event page with a flash message. There is no JavaScript beyond Turbo Drive, which is already installed, and there are no background jobs.

Data flow

1. **Read.** `EventsController#show` also loads `@comments = @event.comments.chronological.includes(:user)`. This adds one query for comments and one for their authors. Guests can read comments, because `show` already allows guests.
2. **Create.** A signed-in person submits the form to `POST /events/:event_id/comments`. `CommentsController#create` builds the comment with `@event.comments.build(comment_params.merge(user: Current.user))`. On success it redirects to `event_path(@event, anchor: "comments")` with the notice "Comment posted." On a validation failure it redirects to the same place with the error as an alert. This keeps the controller as small as `RsvpsController`. Whether to render the form again with the typed text kept is under Outstanding questions.
3. **Delete.** The person clicks `button_to "Delete"`, which sends `DELETE /events/:event_id/comments/:id`. The controller finds the comment through `@event.comments.find(params[:id])` and checks `comment.deletable_by?(Current.user)`. That returns true for the author or for the event's organizer, using `event.organized_by?(user)`. If allowed, the comment is hard-deleted and the controller redirects with the notice "Comment deleted." If not, it redirects with an alert and nothing is deleted.
4. **Guests.** `CommentsController` does not call `allow_unauthenticated_access`, so `require_authentication` sends guests to sign in, the same as for RSVPs. Instead of the form, the page shows a "Sign in to comment" link to `new_session_path`.

Placement

The comments section is a new partial, `app/views/events/_comments.html.erb`, rendered below the RSVP card. The RSVP card sits in the narrow `aside#event-facts`. This plan places the comments as a full-width `section#comments.card` under `.event__layout`, which puts it below the RSVP card on every screen size and keeps long text and links readable. The alternative is to render it inside the aside after the attendees list. That choice is under Outstanding questions.

Legacy coexistence

This change adds a new table only. No existing column is renamed, moved or read differently, and no existing data needs backfilling, so there is nothing to dual-write. Every event starts with zero comments and the section shows an empty state. The one legacy edge case is users whose `name` is `""` (the database default). The view shows a fallback label for them instead of a blank author.

In expand-and-contract terms:

- **Expand:** migration and model.
- **Dual write and backfill:** not applicable.
- **Switch reads:** the controller and view ship.
- **Contract:** none is planned.

Database changes

Phase 1: expand (new table only)

Migration `db/migrate/<timestamp>_create_comments.rb` . It declares the check constraint inside `create_table` , the same way `db/migrate/20260929214624_create_rsvps.rb` declares `rsvps_status_check` . On SQLite, this puts the constraint in the `CREATE TABLE` statement. Adding a constraint to an existing table later would force a table rebuild.

```
class CreateComments < ActiveRecord::Migration[8.1]
  def change
    create_table :comments do |t|
      t.references :event, null: false, foreign_key: true, index: false
      t.references :user, null: false, foreign_key: true
      t.text :body, null: false
      t.timestamps

      t.index %i[ event_id created_at ]
      t.check_constraint "length(trim(body)) > 0 AND length(body) <= 1000", name: "co
    end
  end
end
```

The resulting `comments` table:

Column	Type	Null	Default	Notes
<code>id</code>	integer	not null	auto	primary key
<code>event_id</code>	integer	not null	none	foreign key to <code>events.id</code>
<code>user_id</code>	integer	not null	none	foreign key to <code>users.id</code> ; the author
<code>body</code>	text	not null	none	1 to 1,000 characters; must not be blank
<code>created_at</code>	datetime	not null	none	shown as "posted at"
<code>updated_at</code>	datetime	not null	none	never changes, because editing is out of scope

Indexes:

- `index_comments_on_event_id_and_created_at` on `(event_id, created_at)` . It serves the ordered list on the event page. Its leading column also covers lookups by `event_id` alone, so `t.references :event uses index: false` .
- `index_comments_on_user_id` on `(user_id)` , created by `t.references :user` . It supports `User#comments` and removing a user's comments when the user is deleted.

Foreign keys:

- `comments.event_id → events.id`
- `comments.user_id → users.id`

Neither foreign key uses `on_delete` . Clean-up happens through `Event has_many :comments, dependent: :delete_all` and `User has_many :comments, dependent: :destroy` , which matches how `rsvps` works today.

The 1,000-character limit, in two places

The limit is enforced in the database and in the model, and both must use the same number.

- **Database:** `comments_body_length_check` is `length(trim(body)) > 0 AND length(body) <= 1000` .
- The first part rejects empty bodies and bodies made only of spaces.
- The second part caps the stored value at 1,000 characters. SQLite's `length()` counts characters, not bytes, for text values, so emoji and accented letters count as one character each, the same as in Ruby.
- **Model (`app/models/comment.rb`):** normalizes `:body`, with: `->(body) { body.strip }` and validates `:body, presence: true, length: { maximum: 1000 }` .
- Because the model strips the body before saving, the database checks the value after stripping. That is why the constraint measures `length(body)` and not `length(trim(body))` for the upper bound.
- SQLite's `trim()` removes only spaces, while Ruby's `strip` also removes newlines and tabs. So the model is the stricter check for blank bodies, and the constraint is a backstop for writes that skip validations, such as `insert_all` and `update_column` .
- **Form:** the `text_area` uses `maxlength: 1000` , replacing 2000 in Application changes.
- Browsers count a line break as 1 character for `maxlength` , but submit it as `\r\n` , which is 2 characters on the server. A comment with many line breaks can pass the browser check and still fail the 1,000 limit. The model validation then rejects it with a readable error before the constraint is reached.
- Whether to normalize `\r\n` to `\n` in `normalizes` is added to Outstanding questions.

Other tables

No changes to `events` , `rsvps` , `users` or `sessions` . No column is removed or renamed, so no `ignored_columns` step is needed.

Expand and contract

- **Expand:** this migration.

- **Dual write and backfill:** not needed. The table is new and starts empty.
- **Switch reads:** not needed. No existing read moves to this table.
- **Contract:** none.

Changing the limit later

Raising or lowering 1,000 later takes a new migration that drops and re-adds

`comments_body_length_check` with `remove_check_constraint` and `add_check_constraint`. On SQLite, Rails does this by rebuilding the table. It must ship together with the matching model validation change.

Lowering the limit also needs a check first for existing rows longer than the new limit, because the new constraint would reject them. Settle the number before this migration ships.

Rollback

The migration is reversible: rolling it back drops `comments`. Rolling back after release deletes every comment posted since then. To hide the feature but keep the data, revert the view and controller changes instead.

Application changes

Model: `app/models/comment.rb` (new)

```
class Comment < ApplicationRecord
  belongs_to :event
  belongs_to :user

  normalizes :body, with: ->(body) { body.strip }
  validates :body, presence: true, length: { maximum: 2000 }

  scope :chronological, -> { order(:created_at, :id) }

  def deletable_by?(user)
    user.present? && (user_id == user.id || event.organized_by?(user))
  end
end
```

- Ordering by `created_at, id` matches `Rsvp.in_line_order`, so two comments posted in the same second still come out in a fixed order.

Model: `app/models/event.rb`

- Add `has_many :comments, dependent: :delete_all`, the same dependency as `rsmps`.

Model: `app/models/user.rb`

- Add `has_many :comments, dependent: :destroy`, the same as `rsvps` and `sessions`. Without it, the `comments.user_id` foreign key would block deleting a user. Whether a deleted user's comments should instead stay, shown as anonymous, is under Outstanding questions.

Routes: `config/routes.rb`

```
resources :events, except: :destroy do
  resource :rsvp, only: %i[ create destroy ]
  resources :comments, only: %i[ create destroy ]
end
```

This gives `event_comments_path(event)` (POST) and `event_comment_path(event, comment)` (DELETE).

Controller: `app/controllers/comments_controller.rb` **(new)**

- Requires sign-in for every action (the `Authentication` default).
- `before_action :set_event` with `Event.find(params[:event_id])`, as in `RsvpsController`.
- `rate_limit to: 10, within: 1.minute, only: :create, with: -> { redirect_to event_path(@event, anchor: "comments"), alert: "You're commenting too fast. Try again in a minute." }`, using the Rails 8 built-in rate limiter. The numbers are an assumption.
- `create: @event.comments.build(params.expect(comment: [ :body ]).merge(user: Current.user))`. The `user_id` never comes from `params`. On save it redirects with the notice "Comment posted." Otherwise it redirects with `alert: comment.errors.full_messages.to_sentence`.
- `destroy: comment = @event.comments.find(params[:id])`. Scoping the lookup to the event means the id in the URL can't reach a comment on another event: a mismatch raises `RecordNotFound`, which returns 404. If `comment.deletable_by?(Current.user)`, it calls `comment.destroy`, logs `Rails.logger.info("Comment deleted: comment_id=... event_id=... by_user_id=... by_organizer=...")` and redirects with the notice "Comment deleted." Otherwise it redirects with the alert "You can't delete this comment."
- Whether comments are allowed after `starts_at`: this plan assumes yes, so there is no `rsvps_open?` check. See Outstanding questions.

Controller: `app/controllers/events_controller.rb`

- In `show`, add `@comments = @event.comments.chronological.includes(:user)`. Other actions don't change.

Views

- **New** `app/views/events/_comments.html.erb`: a `section.card#comments` containing:
- The heading `Comments (<%= @comments.size %>)`. `size` uses the loaded records, so there is no extra `COUNT` query.

- An ordered list. Each item has `id=dom_id(comment)` and shows the author name. Blank names fall back to "Gather member", because older users may have `name = ""`. The item shows the post time as `<time datetime="<%= comment.created_at.iso8601 %>">` formatted as `%-d %B %Y · %H:%M`, the same format as `event_time`, and the body. For each comment where `comment.deletable_by?(Current.user)`, it shows `button_to "Delete"`, `event_comment_path(@event, comment)`, `method: :delete`, `class: "button button--ghost"`, `data: { turbo_confirm: "Delete this comment?", turbo_submits_with: "Deleting..." }`.
- The body is rendered with `simple_format(h(comment.body))`. Escaping first means users can't inject markup, including `<a>` tags that `simple_format` would otherwise allow. Pasted URLs appear as plain text. Turning URLs into clickable links is under Outstanding questions, and there is no `rails_autolink` gem in the `Gemfile` today.
- Empty state: "No comments yet. Ask a question or share a link."
- For signed-in users, a `form_with model: [ @event, Comment.new ]` with a `text_area :body`, `required: true`, `maxlength: 2000` and the submit button "Post comment" (`turbo_submits_with: "Posting..."`). For guests, `link_to "Sign in to comment"`, `new_session_path`, `class: "button"`, using `authenticated?` the same way `_rsvp_card.html.erb` does.
- `app/views/events/show.html.erb`: add `<%= render "events/comments" %>` after the closing `</div>` of `.event__layout` and inside the `<article>`.
- `app/assets/stylesheets/application.css`: minimal styles for `.comments` list items (author, muted timestamp, body, delete button). Reuse the existing `card`, `muted`, `small` and `button--ghost` classes.

Policies

There is no policy layer in the app, so this plan doesn't add one. `Comment#deletable_by?` is the single source of the delete rule, used by both the controller and the view.

Jobs

None.

Infrastructure changes

None. There are no new queues, jobs, external services, gems or environment variables.

- **Rate limiting** uses Rails' built-in `rate_limit`, which stores counters in `Rails.cache`. Check that production's cache store is shared across Puma workers and hosts. Otherwise the limit applies per process only. See Outstanding questions.
- **Feature flags**: the app has no flag system (no Flipper or similar in the `Gemfile`). This plan ships without a flag. If the team wants a dark launch, the simplest gate is rendering `events/comments` only when an environment variable such as `COMMENTS_ENABLED` is set, but that is not planned by default.
- **Rollout order**, one deploy per step or combined:
- Migration and `Comment` model, plus the associations on `Event` and `User`. Nothing reads or writes comments yet.

- Routes and `CommentsController` .
- `events/_comments` partial and the change to `EventsController#show` . This makes the feature visible.
- **Rollback:** revert step 3 to hide the feature and keep the data. Only roll back the migration if the data should be discarded.

Work overview

Out of Scope

Replies and threads, editing a comment, email notifications, a moderation queue.

Work items

#	Title	Type	Kind	Estimate	Depends	Issue
T1	Migration: Create comments table	TASK	migration	2	-	#23
T2	Show comments on the event page	STORY	code	5	T1	#24
T3	Post a comment on an event	STORY	code	3	T2	#25
T4	Limit how fast one person can post comments	STORY	code	2	T3	#26
T5	Delete your own comment	STORY	code	3	T3	#27
T6	Let organizers delete any comment on their event	STORY	code	2	T5	#28

Estimated total: 17 points

T1. Migration: Create comments table

Add a new `comments` table that stores one comment per row. Each row links to an event and to the user who wrote it. The table has columns `id` (integer primary key), `event_id` (integer, not null, foreign key to `events.id`), `user_id` (integer, not null, foreign key to `users.id`), `body` (text, not null, no default) and `created_at` / `updated_at` (datetime, not null). The database must protect the body itself. A check constraint named `comments_body_length_check` rejects empty bodies, bodies made only of spaces and bodies longer than 1000 characters. Declare the constraint inside `create_table` , so SQLite puts it in the CREATE TABLE statement and never needs a table rebuild. Add a composite index on `(event_id, created_at)` for the ordered list on the event page, and an index on `user_id` . Neither foreign key cascades on delete. Later tickets clean up comments through model associations, the same way `rsvps` works. This ticket adds no model or application code, so nothing reads or writes the table yet. Its Cucumber scenarios work directly against the database, like `features/pd-1-rsvps-with-a-waitlist/t1.feature` .

ACCEPTANCE CRITERIA

1. After migrating, the comments table exists with NOT NULL columns event_id, user_id, body, created_at and updated_at, and body has no default
2. Inserting a comments row with an empty body fails the check constraint "comments_body_length_check"
3. Inserting a comments row whose body is only spaces fails the check constraint "comments_body_length_check"
4. Inserting a comments row with a 1000-character body succeeds, and a 1001-character body fails the check constraint "comments_body_length_check"
5. A 1000-character body made of multi-byte characters such as emoji or accented letters is accepted, because the limit counts characters and not bytes
6. The index "index_comments_on_event_id_and_created_at" covers event_id, created_at, and the index "index_comments_on_user_id" covers user_id
7. There is no index on event_id alone
8. Inserting a comments row for a missing event or a missing user fails with ActiveRecord::InvalidForeignKey
9. No comments foreign key cascades on delete: deleting an event or a user that has a comments row directly in the database fails with ActiveRecord::InvalidForeignKey
10. Rolling back the migration drops the comments table and leaves the events, users, rsmps and sessions tables unchanged

IMPLEMENTATION NOTES

New file `db/migrate/<timestamp>_create_comments.rb`, using `ActiveRecord::Migration[8.1]`. Follow `db/migrate/20260929214624_create_rsmps.rb`, which declares its check constraint the same way.

```
create_table :comments do |t|
  t.references :event, null: false, foreign_key: true, index: false
  t.references :user, null: false, foreign_key: true
  t.text :body, null: false
  t.timestamps
  t.index %i[ event_id created_at ]
  t.check_constraint "length(trim(body)) > 0 AND length(body) <= 1000", name: "commer
end
```

The limit is 1000. The plan says 2000 in its Application changes section, but its Database section states that 1000 replaces 2000. Every ticket in this set uses 1000. Confirm the number with Ana Kovač before this ticket merges, because changing it later needs another migration and a table rebuild (Outstanding question 3).

Commit the regenerated `db/schema.rb`. Put the Cucumber scenarios in `features/pd-3-comments-on-events/t1.feature` with the tag `@pd-3-t1`. Add table helpers modelled on `features/support/rsmps_table.rb` and `features/step_definitions/rsmps_table_steps.rb`, and optionally a spec like `spec/db/rsmps_table_spec.rb`.

T2. Show comments on the event page

I want to read the questions and links people have left on an event page, oldest first, with who wrote them and when, so that I can find answers without asking in chat again.

Add the `Comment` model and a read-only comments section on the event page. Everyone can see it, including guests.

Model `app/models/comment.rb`:

- `belongs_to :event` and `belongs_to :user`.
- `normalizes :body` strips surrounding whitespace.
- `validates :body, presence: true, length: { maximum: 1000 }`.
- `scope :chronological, -> { order(:created_at, :id) }`.

Associations on existing models:

- `Event : has_many :comments, dependent: :delete_all, inverse_of: :event`.
- `User : has_many :comments, dependent: :destroy`.

In `EventsController#show`, load `@comments = @event.comments.chronological.includes(:user)`.

New partial `app/views/events/_comments.html.erb`:

- A `section.card#comments` with the heading `Comments (N)`. `N` comes from `@comments.size`, so there is no extra `COUNT` query.
- An ordered list of comments. Each item has `id=dom_id(comment)` and shows the author name. When the author's name is blank, it shows "Gather member" instead.
- Each item shows the post time in a `<time datetime=ISO8601>` tag, formatted as `%-d %B %Y · %H:%M` like the `event_time` helper.
- Each item shows the body rendered with `simple_format(h(comment.body))`, so any HTML the user typed appears as plain text.
- When there are no comments, the section shows the empty state "No comments yet. Ask a question or share a link."

Render the partial in `app/views/events/show.html.erb` after the closing `</div>` of `.event__layout` and inside `<article>`. This makes it a full-width section below the RSVP card. Add minimal CSS so long bodies and URLs wrap with `overflow-wrap: anywhere`. Posting and deleting come in later tickets.

ACCEPTANCE CRITERIA

1. A guest opening an event page sees its comments listed oldest first, each with the author's name, the posting time and the body
2. A signed-in person and the organizer see the same list of comments
3. Two comments with the same `created_at` are always listed in id order
4. The heading shows the number of comments, for example "Comments (3)"
5. An event with no comments shows "No comments yet. Ask a question or share a link."

6. The comments section appears below the RSVP card, after the two-column layout
7. A comment whose author has an empty name shows the author as "Gather member"
8. A comment body containing `<script>` or `<a href=...>` markup is shown as literal text, and no script or link element is created
9. Line breaks in a comment body are shown as separate lines
10. No email address appears on the event page when comments are present
11. The number of SQL queries for the event page does not grow with the number of comments
12. Deleting an event that has comments also removes its comments
13. Deleting a user who has comments removes their comments without a foreign key error
14. A comment is invalid without a body, with only whitespace, or with more than 1000 characters, and surrounding whitespace is stripped before saving

IMPLEMENTATION NOTES

Files to add:

- `app/models/comment.rb`
- `app/views/events/_comments.html.erb`
- `spec/models/comment_spec.rb`
- `spec/factories/comments.rb`, a factory associated with `event` and `user`
- `features/pd-3-comments-on-events/t2.feature`, tagged `@pd-3-t2`
- `features/step_definitions/comment_steps.rb`

Files to edit:

- `app/models/event.rb`
- `app/models/user.rb`
- `app/controllers/events_controller.rb` (show only)
- `app/views/events/show.html.erb`
- `app/assets/stylesheets/application.css`
- `spec/models/event_spec.rb` and `spec/models/user_spec.rb`, for the dependent-delete specs
- `spec/requests/events_spec.rb`

Do not add `deletable_by?` or a Delete button yet. T5 adds them.

The no-N+1 check depends on the event association. `inverse_of: :event` makes `comment.event` reuse the already-loaded `@event`, which T5 relies on. Test it by counting SQL queries for `show` with 1 comment and with 5 comments, and assert the count is the same.

Never use `raw` or `html_safe` on the body. Reuse the existing `card`, `muted` and `small` CSS classes.

When this ticket merges alone, the page shows the empty state with no way to post yet. Release T2 and T3 in the same deploy if that intermediate state is unwanted. Placement is Outstanding question 1: this ticket assumes full width.

Touches: comments, users

T3. Post a comment on an event

I want to post a question or a link on an event page, so that the organizer and other attendees can answer it where the next person will see it.

Let signed-in people post comments.

Routes: add `resources :comments, only: %i[ create ]` inside `resources :events` in `config/routes.rb`. T5 adds `:destroy` later.

New `app/controllers/comments_controller.rb`:

- Keeps the default `require_authentication`, so guests are sent to sign in.
- `before_action :set_event` with `Event.find(params[:event_id])`.
- `create` builds `@event.comments.build(params.expect(comment: [ :body ]).merge(user: Current.user))`. The author always comes from the session, never from params.
- On success, it redirects to `event_path(@event, anchor: "comments")` with the notice "Comment posted."
- On a validation failure, it redirects to the same place with `alert: comment.errors.full_messages.to_sentence`.

In `_comments.html.erb`, signed-in users see a `form_with` model: `[ @event, Comment.new ]` below the list. The form has a `text_area :body, required: true, maxlength: 1000` and a "Post comment" submit button with `data: { turbo_submits_with: "Posting..." }`. Guests see `link_to "Sign in to comment", new_session_path, class: "button"` instead, using `authenticated?` the same way `_rsvp_card.html.erb` does.

Comments can be posted before and after the event starts. There is no `rsmps_open?` check.

ACCEPTANCE CRITERIA

1. A signed-in person posts a question and sees it at the bottom of the comments list with their name and the posting time
2. After posting, the person is taken back to the comments section of the event page and sees "Comment posted."
3. A guest sees a "Sign in to comment" link and no comment form
4. A guest who sends a comment POST directly is redirected to sign in and no comment is created
5. A forged `user_id` in the submitted params is ignored and the comment belongs to the signed-in person
6. Submitting a blank or whitespace-only body creates no comment and shows an error alert
7. Submitting a body longer than 1000 characters creates no comment and shows an error alert
8. A person can comment on an event that has already started
9. The comment text area limits input to 1000 characters in the browser

IMPLEMENTATION NOTES

Files to add:

- `app/controllers/comments_controller.rb`, modelled on `app/controllers/rsmps_controller.rb`

- `spec/requests/comments_spec.rb` , in the style of `spec/requests/rsvps_spec.rb` , using `spec/support/authentication_helpers.rb`
- `features/pd-3-comments-on-events/t3.feature` , tagged `@pd-3-t3`

Files to edit:

- `config/routes.rb`
- `app/views/events/_comments.html.erb`

The form posts to `event_comments_path(@event)` .

A body with many line breaks can pass the browser's `maxlength` but fail the model check. Browsers count a line break as 1 character but submit it as `\r\n` , which is 2. The model validation catches this with a readable alert (Outstanding questions 3 and 10).

Rate limiting comes in T4.

Touches: comments

T4. Limit how fast one person can post comments

I want the app to stop one account from flooding an event with comments, so that the conversation stays readable.

Add the Rails 8 built-in rate limiter to `CommentsController#create`: `rate_limit to: 10, within: 1.minute, only: :create, with: -> { redirect_to event_path(@event, anchor: "comments") , alert: "You're commenting too fast. Try again in a minute." }`. Once a person goes over the limit, their further posts in that minute are rejected, and no comment is saved. The limiter stores its counters in `Rails.cache` .

ACCEPTANCE CRITERIA

1. A signed-in person can post 10 comments within one minute
2. The 11th comment within one minute is not saved, and the person sees "You're commenting too fast. Try again in a minute." on the event page
3. After the minute has passed, the same person can post again

IMPLEMENTATION NOTES

Edit `app/controllers/comments_controller.rb` . `set_event` must run before the rate limiter's `with: block`, so declare `before_action :set_event before rate_limit` .

Tests need a real cache store. Use `ActiveSupport::Cache::MemoryStore` in the spec, and clear it between examples. Use `travel` to move past the minute.

Add a request spec to `spec/requests/comments_spec.rb` and scenarios to `features/pd-3-comments-on-events/t4.feature` , tagged `@pd-3-t4` .

Check that production's `config.cache_store` is shared across Puma workers and hosts. Otherwise the limit applies per process only (Outstanding question 8).

Touches: comments

T5. Delete your own comment

I want to delete a comment I posted, so that I can remove a question that is no longer needed or a link I shared by mistake.

Let authors delete their own comments.

Model: add `Comment#deletable_by?(user)` . In this ticket it returns true when `user.present? && user_id == user.id` . T6 extends it for organizers.

Routes: extend to `resources :comments, only: %i[ create destroy ]` .

`CommentsController#destroy` :

- Finds the comment with `@event.comments.find(params[:id])` , so an id belonging to another event returns 404.
- If `comment.deletable_by?(Current.user)` , it calls `comment.destroy` (a hard delete).
- It then logs `Rails.logger.info("Comment deleted: comment_id=... event_id=... by_user_id=... by_organizer=...")` and redirects to `event_path(@event, anchor: "comments")` with the notice "Comment deleted."
- Otherwise it redirects to the same place with the alert "You can't delete this comment." and deletes nothing.

In `_comments.html.erb` , show a Delete button only on comments where `comment.deletable_by?(Current.user)` : `button_to "Delete", event_comment_path(@event, comment), method: :delete, class: "button button--ghost", data: { turbo_confirm: "Delete this comment?", turbo_submits_with: "Deleting..." }` .

ACCEPTANCE CRITERIA

1. The author of a comment sees a Delete button on their own comment
2. After confirming "Delete this comment?", the author's comment disappears from the list and they see "Comment deleted."
3. A signed-in person sees no Delete button on comments written by others
4. A signed-in person who sends a delete request for someone else's comment is shown "You can't delete this comment." and the comment remains
5. A guest who sends a delete request is redirected to sign in and the comment remains
6. A delete request that uses a comment id from a different event returns not found and deletes nothing
7. Deleting a comment writes a "Comment deleted:" log line with the comment, event and user ids
8. The number of SQL queries for the event page still does not grow with the number of comments when Delete buttons are shown

IMPLEMENTATION NOTES

Files to edit:

- `app/models/comment.rb`
- `app/controllers/comments_controller.rb`
- `config/routes.rb`

- `app/views/events/_comments.html.erb`
- `app/assets/stylesheets/application.css` , for the delete button placement
- `spec/models/comment_spec.rb` , for `deletable_by? : true` for the author, `false` for another user, `false` for `nil`
- `spec/requests/comments_spec.rb`
- `spec/requests/events_spec.rb`

Add scenarios in `features/pd-3-comments-on-events/t5.feature` , tagged `@pd-3-t5` . The confirm dialog needs a JavaScript-capable driver, or scenarios that accept the confirm.

In this ticket, `by_organizer` in the log line is always `event.organized_by?(Current.user)` .

Never rely on the button's visibility alone. The server-side check is the rule.

Touches: comments

T6. Let organizers delete any comment on their event

As an event organizer, I want to delete any comment on my event, so that I can remove spam or misleading information.

Extend `Comment#deletable_by?(user)` so it also returns `true` when `event.organized_by?(user)` . The full rule becomes `user.present? && (user_id == user.id || event.organized_by?(user))` . Because the controller and the view both use this one predicate, the organizer gets the Delete button and the server permits the delete. The log line records `by_organizer=true` when the organizer deletes someone else's comment. Organizers of other events get no extra rights.

ACCEPTANCE CRITERIA

1. The organizer sees a Delete button on every comment on their event, including comments by other people
2. The organizer deletes another person's comment after confirming, and it disappears from the list with "Comment deleted."
3. An organizer of one event cannot delete a comment on another event, even by putting that comment's id in their own event's URL, which returns not found
4. An organizer of another event sees no Delete button on this event's comments, and a direct delete request is refused with "You can't delete this comment."
5. A non-organizer still sees Delete buttons only on their own comments
6. When the organizer deletes someone else's comment, the "Comment deleted:" log line includes `by_organizer=true`

IMPLEMENTATION NOTES

Edit `app/models/comment.rb` . `Event#organized_by?` is the app's only organizer check, so reuse it rather than comparing `organizer_id` directly.

`comment.event` must reuse the loaded `@event` through `inverse_of: :event` , added in T2, so the query count stays flat.

Add model specs for organizer true and other user false, request specs in

`spec/requests/comments_spec.rb` for the organizer and the cross-event 404, and scenarios in

`features/pd-3-comments-on-events/t6.feature` , tagged `@pd-3-t6` .

An optional "Organizer" badge is out of scope (Outstanding question 7).

Touches: comments, events

Risks

- **Stored XSS or phishing markup in comment bodies.** Mitigation: render with `simple_format(h(comment.body))` so all HTML is escaped, and never use `raw` or `html_safe` . A request or system test posts `<script>` and `<a href=...>` bodies and asserts they appear as text.
- **Someone deletes another person's comment through a crafted request.** Mitigation: find the comment through `@event.comments.find` , check `Comment#deletable_by?` on the server, never rely on the button's visibility alone, and cover the rule with request specs for author, organizer, another signed-in user and a guest.
- **Spam or flooding by signed-in accounts.** Mitigation: `rate_limit` on `create` , a 2000-character limit enforced in the model and by a database check constraint, and organizers can delete any comment on their event. A moderation queue is out of scope.
- **Deleting a user or event fails because of the new foreign keys.** Mitigation: `Event` has_many :comments, dependent: :delete_all and `User` has_many :comments, dependent: :destroy , with model specs that delete a user and an event that have comments.
- **Blank author names for legacy users (`users.name` defaults to `""`).** Mitigation: a fallback label in the view, covered by a test.
- **Page layout: long URLs or long comments break the layout.** Mitigation: CSS `overflow-wrap: anywhere` on comment bodies, placement in a full-width section, and QA with a 2000-character body and a long URL.
- **The page slows down on events with many comments.** Mitigation: the composite index and `includes(:user)` . Pagination is not planned. See Performance and Outstanding questions.
- **A guest submits a comment and loses it.** A guest POST redirects to sign-in, and `request_authentication` stores the POST URL as `return_to` , which has no GET route. This is the same existing behaviour as RSVPs. Mitigation: guests never see the form. They see a "Sign in to comment" link.

Performance

- **Event page (`EventsController#show`)** adds 2 queries: `SELECT comments WHERE event_id = ? ORDER BY created_at, id` and `SELECT users WHERE id IN (...)` through `includes(:user)` . The heading count uses `@comments.size` on the loaded records, so it adds no `COUNT` query.
- **N+1 risks:**
- `comment.user.name` is preloaded.
- `comment.deletable_by?(Current.user)` calls `event.organized_by?` . `comment.event` would trigger one query per comment unless the association points back to the already-loaded event. Either declare `has_many :comments, inverse_of: :event` , which Rails usually infers for this

simple association, or pass `@event` in. A test asserts the query count stays flat as the number of comments grows.

- **Indexes:** the `(event_id, created_at)` index serves the filtered, ordered list. The `user_id` index serves user deletion.
- **Size:** this is a new table and needs no backfill, so there are no batch sizes to set. A busy event could build up hundreds of comments. All of them render on the page because pagination is not planned. That is acceptable at current scale, but it is listed as an Outstanding question with a suggested threshold (for example, show the latest 200 and link to more).
- **Writes:** one `INSERT` or `DELETE` per request, with no locking. `Event#rsvp` uses `lock!`, but comments don't compete for seats, so they don't need it.
- **Events index page:** no change. Comment counts are not shown there.

Security

- **Authentication:** reading is public. `EventsController#show` already allows guests, and comments are meant to help the next visitor. Creating and deleting require a session, because `CommentsController` keeps the default `require_authentication`.
- **Authorization:** `Comment#deletable_by?(user)` allows the author (`user_id == user.id`) or the event's organizer (`event.organized_by?(user)`). The controller enforces this server-side on every `destroy`. The lookup is scoped with `@event.comments.find(params[:id])`, so a comment id from another event returns 404. The author is always `Current.user` and is never read from params. Strong params allow only `body` (`params.expect(comment: [ :body ])`).
- **Input validation:** the body is stripped and must be present, at most 2000 characters. This is enforced by the model and by the `comments_body_length_check` constraint.
- **Output encoding:** `simple_format(h(body))` escapes all user HTML. There are no `raw` or `html_safe` calls. If auto-linking is added later, it must only produce `http` and `https` links with `rel="nofollow noopener ugc"`.
- **CSRF:** the form and delete buttons use `form_with` and `button_to`, which include Rails' authenticity token. The existing `ApplicationController` protection applies.
- **Data exposure:** only `users.name` and the timestamp are shown next to a comment. Email addresses must never appear. This matches the existing `@pd-1-t4 @ac-6` scenario, which asserts that the event page contains no email addresses, and it is extended to comments. Comments are public, so the UI copy should make clear that anyone can read them.
- **Abuse:** `rate_limit` on `create`. A deleted comment is hard-deleted, so no audit trail remains beyond the log line.

Risk Level: MEDIUM. This is the first feature that shows free text from one user to other users, including guests, so escaping and the delete permission check must be right, although the data involved isn't sensitive.

Monitoring

- **Errors:** watch for new exceptions from `CommentsController`, especially `ActiveRecord::RecordNotFound` spikes on `destroy`, which could mean id probing, and `ActiveRecord::StatementInvalid` from the check constraint, which would mean the model

validation and the constraint disagree. Watch for `ActiveRecord::InvalidForeignKey` when users or events are deleted.

- **Logs:** search for the `Comment deleted:` log line to see how often organizers delete others' comments, a signal of spam or abuse. Also track how often requests are rate-limited on `POST /events/:id/comments`.
- **Performance:** response time of `EventsController#show` before and after release, and the number of SQL queries per request in logs. It should rise by exactly 2.
- **Usage metrics (manual queries, since there is no analytics tool in the repo):** `Comment.where(created_at: 1.week.ago..).count` and the number of distinct events and authors with comments, to judge whether questions are moving out of chat as intended.
- **Jobs:** none to monitor.

Outstanding questions

1. **Placement:** "Below the RSVP card" could mean inside the narrow sidebar (`aside#event-facts` , after the Edit link and attendees list) or as a full-width section under the page's two columns. This plan assumes full width. Design needs to confirm before the UI phase ships.
2. **Commenting after the event:** can people comment on events that have already started or ended, when `rsvps_open?` is false? This plan assumes yes.
3. **Maximum length:** is 2000 characters right? The number goes into both the model validation and the database check constraint, so it must be settled before the migration ships.
4. **Links:** should URLs in a comment become clickable, or stay plain text? Clickable links need a small helper or a new gem, plus a review of the link attributes.
5. **Deleted users:** when a user account is deleted, should their comments be deleted too (the plan's assumption, matching RSVPs) or kept with an anonymous author? Keeping them needs `user_id` to be nullable.
6. **Timestamp format and time zone:** absolute ("30 September 2026 · 14:12") or relative ("5 minutes ago")? Which time zone is `config.time_zone` set to in production? This wasn't checked.
7. **Organizer label:** should the organizer's own comments be marked, for example with an "Organizer" badge, so answers stand out?
8. **Rate limit:** is 10 comments per minute per person right, and is production's `Rails.cache` store shared across processes so the limit applies across the whole app?
9. **Volume:** is showing every comment on the page acceptable, or should the list be capped or paginated above some number?
10. **Failed submissions:** should a failed comment (blank or too long) show the form again with the typed text kept, or is a redirect with an alert acceptable given the browser's `required` and `maxlength` checks?

Testing

Model: `spec/models/comment_spec.rb`

- Valid with an event, a user and a body. Invalid without a body, with only whitespace, or longer than 2000 characters.
- `normalizes` strips surrounding whitespace.
- `chronological` orders by `created_at` then `id`, including two comments with the same timestamp.
- `deletable_by?` is true for the author, true for the event's organizer, false for another user and false for `nil`.
- The database check constraint rejects an empty body when validations are bypassed (`insert_all` or `update_column`).

Models: `spec/models/event_spec.rb` **and** `spec/models/user_spec.rb`

- Deleting an event with comments removes its comments.
- Deleting a user with comments removes their comments and doesn't raise a foreign key error.

Factory: `spec/factories/comments.rb`

- A `comment` factory associated with `event` and `user`.

Request: `spec/requests/comments_spec.rb`

These follow the style of `spec/requests/rsvps_spec.rb`.

- **Guest:** POST and DELETE redirect to `new_session_path` and change nothing.
- **Signed in:** POST creates a comment owned by `Current.user` and redirects to `event_path(event, anchor: "comments")` with the notice "Comment posted." A forged `user_id` in the params is ignored.
- **Validation:** a blank body creates nothing and redirects with an alert.
- **Author delete:** the author can delete their own comment.
- **Organizer delete:** the organizer can delete someone else's comment on their event.
- **Blocked deletes:**
 - Another signed-in user can't delete someone else's comment. They are redirected with an alert and the comment count is unchanged.
 - An organizer of event A can't delete a comment on event B, even by putting B's comment id in A's URL (404).
- **Rate limit:** the 11th POST within a minute is rejected.

Request: `spec/requests/events_spec.rb`

- `GET /events/:id` lists comments oldest first, with author names and times, for a guest, a signed-in user and the organizer.
- The page shows the form only when signed in, and the "Sign in to comment" link otherwise.
- Delete buttons appear only on comments the viewer may delete.

- Escaping: a body containing `<script>` or `<a href=...>` is rendered as escaped text.
- **Legacy data:** a comment by a user whose `name` is `""` shows the fallback label.
- No email address appears on the page when comments are present.
- The query count for `show` doesn't grow with the number of comments (no N+1).

System and acceptance: Cucumber, in `features/pd-N-comments-on-events/`

The repo uses Cucumber features instead of `spec/system` .

- A signed-in person posts a question and sees it at the bottom of the list with their name and time.
- A guest sees existing comments and a "Sign in to comment" link, but no form.
- An author deletes their own comment after confirming.
- The organizer deletes another person's comment.
- A non-organizer sees no delete button on others' comments.
- An event with no comments shows the empty state.
- The comments section appears below the RSVP card.
- A long URL doesn't overflow the layout (manual QA check).

Sign-off

Role	Decision	By	When
Review	approved	Ivan Blažević <ivan.blazevic@rubycode.co>	30 Sep 2026 12:18